نماذج عمليات البرمجيات

- **نموذج الشلال (The waterfall model) :**
  - **مراحله**
  - **عيوبه**
  - **مشاكل نموذج الشلال**
- **النموذج الحلزوني (Spiral model) :**
  - **قطاعات نموذج الحلزون**
- **نموذج الآجايل (agile model) :**
  - **مبادئ الآجايل الإثنى عشر**
  - **أساليب وطرق الأجايل**
  - **مميزات منهجية الآجايل**
  - **الفرق بين المنهجية الرشيقة (Agile) ونموذج الشلال (Waterfall Method)**
- **النموذج السريع (Rapid Application Development) & (Rapid model) :**
  - **ما هي منهجية (RAD)**
  - **المراحل الرئيسية لمنهجية RAD**
  - **متى تستخدم منهجية RAD**
  - **مزايا منهجية RAD**
  - **عيوب نموذج RAD**

نماذج أخرى لعمليات البرمجيات

- **نموذج التطوير التدريجي.(Evolutionary development)**
  - **مشاكل التطوير الارتقائي.**
  - **قابلية تطبيق التطوير الارتقائي.**
- **نموذج التطوير المبنى على إعادة الاستخدام.(Reuse-oriented development)**
  - **مراحل العمليات.(Process stages) .**
  - **خطوات العمليات (process steps) .**

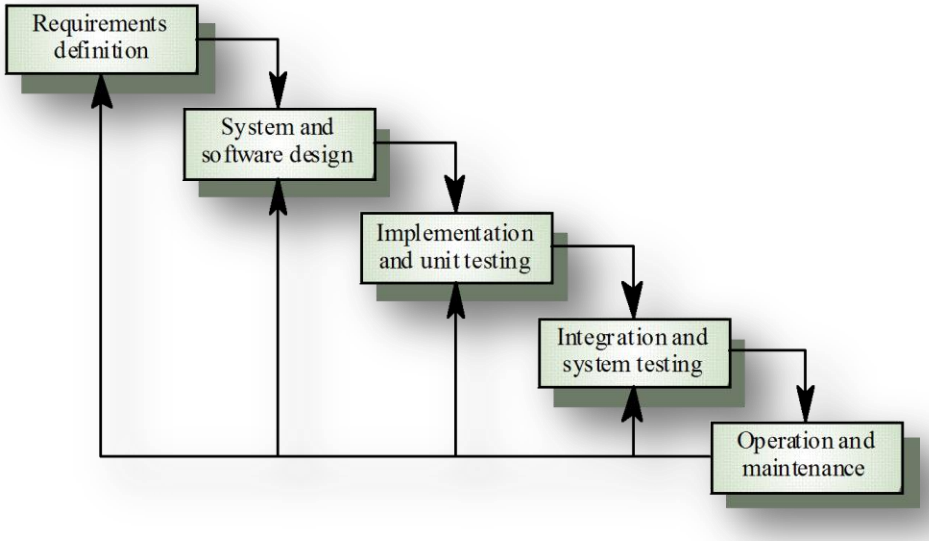
نماذج عمليات البرمجيات

نموذج العملية البرمجية (Software process model) :
هو تمثيل مجرد لوصف العملية من منظور معين . ومن النماذج العامة لعمليات البرمجيات :

- **نموذج الشلال (The waterfall model) :**
وهى عبارة عن مراحل واضحة المعالم منفصلة لوضع المواصفات والتطوير .

❖ **مراحله :**
- تعريف وتحليل وتحديد المتطلبات (Requirements analysis and definition) .
- تصميم النظام وتصميم البرمجيات (System and software design) .
- تنفيذ واختبار وحدات النظام (Implementation and unit testing) .
- تجميع النظام واختباره (Integration and system testing) .
- عمل النظم وصيانته (Operation and maintenance) .

Waterfall model

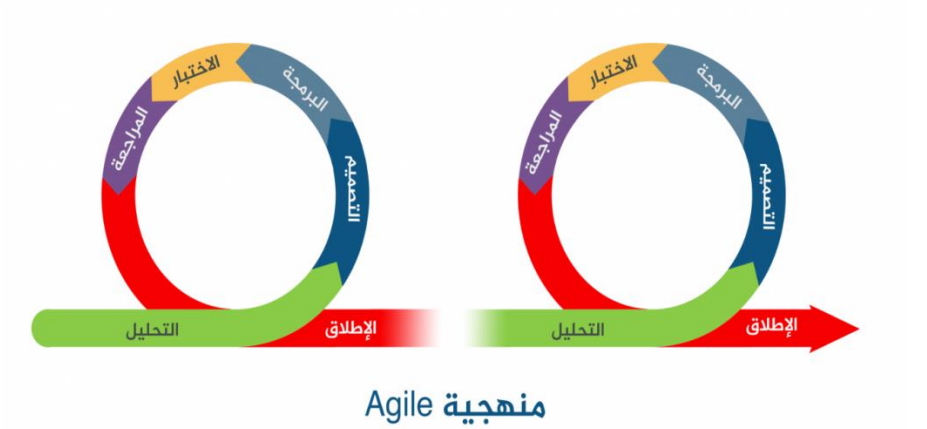


❖ **عيوبه :** من عيوب نموذج الشلال صعوبة تقدير وتكييف التغييرات أثناء العملية .
❖ **مشاكل نموذج الشلال :** التقسيم غير المرن للمشروع إلى مراحل منفصلة ، يزيد من صعوبة الاستجابة عند تغيير متطلبات المستهلك ، لهذا يصبح هذا النموذج مفيداً فقط ومرغوباً عند التفهم الكامل للمتطلبات وقلة التغييرات فيها إلى الحد الأدنى .

- **نموذج الآجايل (agile model) :**

- الآجايل تعريفها : هي لغة الرشاقة والمرونة وترمز آجايل إلى سرعة التعامل مع المتغيرات بسلاسة .

وهي منهجية أو **مبادئ** في عملية تطوير البرمجيات يتم من خلالها إنشاء وتطوير البرمجيات من خلال جهود التعاون داخل فريق عمل معين ومتعدد الوظائف ذو تنظيم ذاتي .



❖ **مبادئ الآجايل الإثنى عشر :**

1. إرضاء العميل عن طريق التسليم المبكر والمتواصل لبرمجيات ذات قيمة.
2. الترحيب بتغيير المتطلبات ولو في مراحل متقدمة من التطوير.
3. تسليم برمجيات صالحة للاستعمال على فترات منتظمة ، من أسبوعين إلى شهرين.
4. يجب أن يعمل كلاً من المهنيين (العارفين بالمِهنة) والمطورين معاً بشكل يومي خلال فترة المشروع.
5. الاعتماد على أفراد متحمسين مع توفير البيئة المناسبة والدعم اللازم ومنحهم الثقة من أجل إنجاز العمل.
6. أكثر الطرق فاعلية وتأثير لفريق التطوير هي التخاطب وجهاً لوجه بين أفراده.
7. البرمجيات الصالحة للاستعمال هي المقياس الرئيسي للتقدم.
8. مناهج الأجايل تشجع التطوير المستدام.
9. الاهتمام المستمر بالتفوق التقني والتصميم الجيد يعزز درجة الأجايل.
10. البساطة – (فن تقليص الأعمال غير الضرورية) - أساسية.
11. أفضل البنيات والمواصفات والتصميمات تنتج من فرق العمل ذاتية التنظيم.
12. فريق العمل يراجع على فترات منتظمة لكي يصبح أكثر فاعلية ثم يدقق ويضبط سلوكه وفقا لذلك.

❖ **أساليب وطرق الأجايل :**

- تعتمد الأجايل على الاجتماعات القصيرة لفريق العمل حيث يتم فيها مناقشة المستجدات والمشاكل البرمجية التي تمت مواجهتها أثناء العمل.
- يقوم كل من أعضاء الفريق بعد ذلك بحل هذه المشاكل على حدى.
- يعمل فريق العمل على استخدام لغات البرمجة والحلول البرمجية التي تتناسب مع معرفتهم.
- يقوم فريق العمل بنمذجة المخططات بطريقة مبسطة و يتم تصميم المخططات ذات الأهمية بالبرامج المخصصة بالمخططات التصميمية.

- تعمد عملية بناء البرمجيات على تكريس معظم الوقت لإنشاء البرامج بدلاً من إعداد التقارير الخاصة بتوثيق هذه البرامج.
- يعتمد المبرمجين في منهجية الأجايل على اتباع الطرق البسيطة أولاً لحل المشاكل البرمجية.
- يعتمد المبرمجون في منهجية الأجايل على التركيز على الوظائف الأساسية في الأنظمة البرمجية حيث يتم الاعتماد عليها أكثر من بقية الأجزاء الأخرى في تلك الأنظمة البرمجية.
- يعمل المبرمجون على تلقي التغذية الراجعة من مستخدمين النظام بشكل دوري مما يؤدي إلى تطوير البرنامج بشكل سريع.

❖ **مميزات منهجية الآجايل :**

من أهم ما تميزت به هذه المنهجية هو (رشاقتها) حيث برزت عدة ميزات منها :

- المقدرة على الاستجابة لأي تغييرات في الخطة المقترحة لبناء البرمجيات خلال أي مرحلة من مراحل تطوير المشروع :

  حيث يتم تقسيم المشروع كاملاً إلى جزئيات صغيرة وكل جزئية تعتبر مشروع صغير يمكن تقديمه للزبون وهذا ما يكسب الزبون الرضا كما يمكنه إبداء آرائه واقتراحاته حول كل مرحلة من مراحل المشروع .
- تعزيز التفاعلات الفردية والجماعية للفريق سواء على مستوى العمليات أو الأدوات :

  وهنا يظهر الاهتمام بالفريق ومعنوياته وخبراته أكثر من الاهتمام بالأدوات المستخدمة كما تتم المشاركة بالمعارف والأدوات المتوفرة لإنجاز المشروع.
- تسهيل التفاوض مع الزبون وقدرته على مراقبة التطورات القائمة على المشروع :

  غالباً الزبون يملك نظرة ضبابية تجاه المشروع في بداية الأمر .. وهذا أمر طبيعي ولكنه غالبا ما كان يسبب العديد من المشاكل في المنهجيات القديمة حيث يكون المنتج في نهاية الأمر مخالف لتوقعات الزبون ، وهذا ما قامت بحلّه هذه المنهجية بحيث كما ذكرنا سابقا يتم تسليم المشروع على مراحل مما يضع الزبون في صورة العمل خطوة بخطوة .

- ورغم اتجاه نسبة كبيرة من مهندسي البرمجيات نحو منهجية ال Agile في تطوير المشاريع إلا أنه لا بد من وجود بعض التفاصيل التي لا يزال يتفوق بها نموذج الشلال وهنا مقارنة بين النموذجين.

❖ **الفرق بين المنهجية الرشيقة (Agile) ونموذج الشلال (Waterfall Method) :**

Agile	Waterfall
تقترح الطريقة الرشيقة نهجًا تدريجيًا وتكراريًا لتصميم البرمجيات	يتطور تطوير البرنامج بالتتابع من نقطة البداية إلى نقطة النهاية.
يعتبر نموذج Agile غير منظم مقارنة بنموذج الشلال	نموذج الشلال أكثر أمانا بما أنه موجه لاتباع خطة ثابتة
يمكن إنجاز المشاريع الصغيرة بسرعة كبيرة بينما المشاريع الكبيرة فيصعب تقدير الزمن المتوقع لإنجازها تبعا للتغييرات في خطة التطوير	يمكن تقدير الزمن الكلي لإنجاز المشاريع بسهولة
يمكن إصلاح الأخطاء أثناء مرحلة التطوير والتحقق من فعالية المشروع خطوة بخطوة	لا يمكن القيام بعمليات الاختبار والصيانة حتى نهاية المشروع وقد يضطر الأمر أحيانا لإعادة بناء المشروع من الصفر
عملية التطوير هنا تكون بمثابة عملية تكرارية لكل جزئية من المشروع وتأخذ من (2-4) أسابيع وتدعى (**Sprint**) أما عملية التخطيط لكل مرحلة فتأخذ وقت أقل بكثير	تأخذ عملية التطوير مدة طويلة نسبياً وكل جزئية من المشروع تحمل تفاصيل للمرحلة القادمة
التوثيق أقل أهمية من عملية التطوير	يأخذ التوثيق أهمية قصوة
يتطلب اتصال وثيق مع المطورين وتحليل المتطلبات والتخطيط معا	لا يتدخل المطورين في عملية التخطيط وتحليل المتطلبات مما يعطي تأخير زمني في العمل

- **بعض المشاكل التي تواجهها أثناء استخدام نموذج الشلال (Waterfall Model).**

1- **صعوبة تعديل المتطلبات:** نظرًا لأن نموذج الشلال يعتمد على تحديد جميع المتطلبات في مرحلة مبكرة وثابتة، فإن تعديل المتطلبات في وقت لاحق قد يكون صعبًا ومكلفًا.
2- **نقص التواصل:** قد يحدث نقص في التواصل بين فرق التطوير المختلفة نتيجة للتقدم التسلسلي في المراحل، مما قد يؤدي إلى سوء التفاهم وزيادة احتمالية حدوث أخطاء.
3- **صعوبة التكيف مع التغييرات:** في حالة تغيرات في المتطلبات أو الظروف، قد يكون من الصعب تكييف عملية التطوير والتغييرات المطلوبة، مما يمكن أن يؤدي إلى تأخير في المشروع وزيادة التكاليف.
4- **عدم قدرة العميل على رؤية المنتج النهائي إلا في وقت متأخر:** نظرًا لطبيعة التسلسلية لنموذج الشلال، قد لا يكون بإمكان العميل رؤية المنتج النهائي إلا في مرحلة متأخرة، مما يزيد من احتمالية عدم تلبية توقعات العميل .
5- **صعوبة التعامل مع المشاكل التقنية المفاجئة:** في حالة ظهور مشاكل تقنية غير متوقعة خلال مرحلة معينة، قد يكون من الصعب التعامل معها بسرعة نظرًا للطبيعة التسلسلية لنموذج الشلال .

- **أهم الفروقات الرئيسية بين نموذج الآجايل (Agile Model) ونموذج الشلال (Waterfall Model)؟**

1- **التوجه العام:**
- **نموذج الآجايل** :يتميز بالتوجه نحو التعاون الوثيق بين فرق التطوير والعملاء، والتفاعل المستمر والتكيف مع التغييرات.
- **نموذج الشلال** :يعتمد على التسلسل الزمني للمراحل، حيث يجب إكمال كل مرحلة قبل الانتقال إلى المرحلة التالية.

2- التخطيط والتنفيذ:
- **نموذج الآجايل:** يتم التخطيط بشكل مرن وتنفيذ التغييرات بسرعة وفعالية خلال دورات التطوير القصيرة.
- **نموذج الشلال:** يتم التخطيط بشكل شامل في بداية المشروع ويصعب تغيير المتطلبات في وسط عملية التطوير.

3- التحكم بالجودة:
- **نموذج الآجايل:** يتم التحكم بالجودة من خلال التحقق المستمر واختبارات الوحدات المتكررة خلال دورات التطوير.
- **نموذج الشلال:** يتم التحقق من الجودة في نهاية كل مرحلة، مما قد يزيد من مخاطر اكتشاف الأخطاء في وقت متأخر.

4- التعامل مع التغييرات:
- **نموذج الآجايل:** يتم التعامل بشكل إيجابي مع التغييرات والمتطلبات الجديدة خلال عملية التطوير.
- **نموذج الشلال:** يصعب تغيير المتطلبات بعد بدء عملية التطوير، مما قد يؤدي إلى تأخيرات وزيادة في التكاليف.

5- التواصل والتعاون:
- **نموذج الآجايل:** يشجع على التواصل المباشر والتعاون الوثيق بين أفراد الفريق والعملاء.
- **نموذج الشلال:** يعتمد على التواصل المكتوب بشكل أساسي وقد يكون التفاعل مع العملاء أقل.

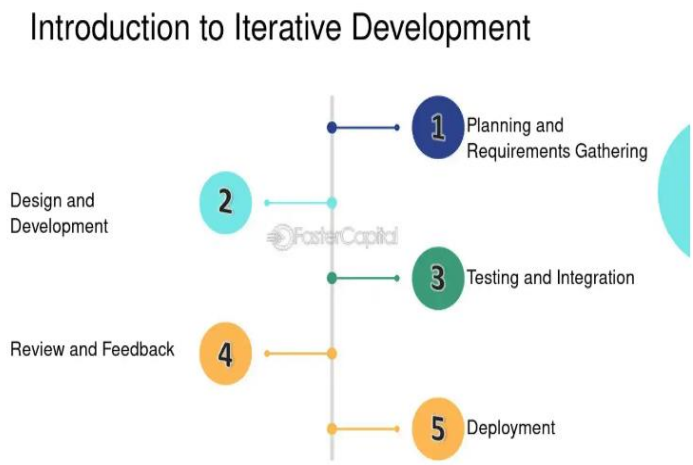
- **النموذج التكراري:(Iterative Model) :**

هو منهجية في تطوير البرمجيات تعتمد على تكرار مراحل عملية التطوير بشكل دوري، مما يسمح بتحسين مستمر وتعديل المتطلبات بناءً على الملاحظات.

الخصائص الرئيسية للنموذج التكراري:

1. **تكرار المراحل:**
   - يتم تنفيذ كل مرحلة من مراحل التطوير (مثل التحليل، التصميم، التنفيذ، والاختبار) عدة مرات، مع تحسينات وتعديلات في كل تكرار.
2. **تطوير تدريجي:**
   - يبدأ المشروع بنموذج أولي بسيط يتم تطويره وتعديله بناءً على الملاحظات من أصحاب المصلحة والمستخدمين حتى الوصول إلى المنتج النهائي.
3. **التفاعل مع العملاء:**
   - يشمل تفاعلًا مستمرًا مع العملاء للحصول على ملاحظاتهم في كل تكرار، مما يساعد في تحسين النظام وفقًا لاحتياجاتهم.
4. **المرونة:**
   - يسمح بإجراء تغييرات على المتطلبات في أي مرحلة من مراحل التطوير، مما يجعله مناسبًا للمشاريع ذات المتطلبات غير الثابتة.

مراحل النموذج التكراري:



1. **التخطيط** :
   - تحديد الأهداف والمتطلبات الأساسية.
2. **التصميم:**
   - تطوير تصميم أولي للنظام.
3. **التنفيذ:**
   - بناء النظام بناءً على التصميم.
4. **الاختبار:**
   - اختبار النظام لتحديد الأخطاء والمشاكل.
5. **التقييم:**
   - مراجعة النتائج وجمع الملاحظات من المستخدمين.
6. **تعديل:**
   - إجراء التعديلات اللازمة بناءً على الملاحظات، ثم الانتقال إلى التكرار التالي.

الاستخدامات:

- يُفضل النموذج التكراري في المشاريع التي تتطلب تفاعلًا مستمرًا مع العملاء أو عندما تكون المتطلبات غير واضحة في البداية.
- يُستخدم بشكل واسع في تطوير البرمجيات المعقدة حيث تحتاج الفرق إلى المرونة في التغييرات.

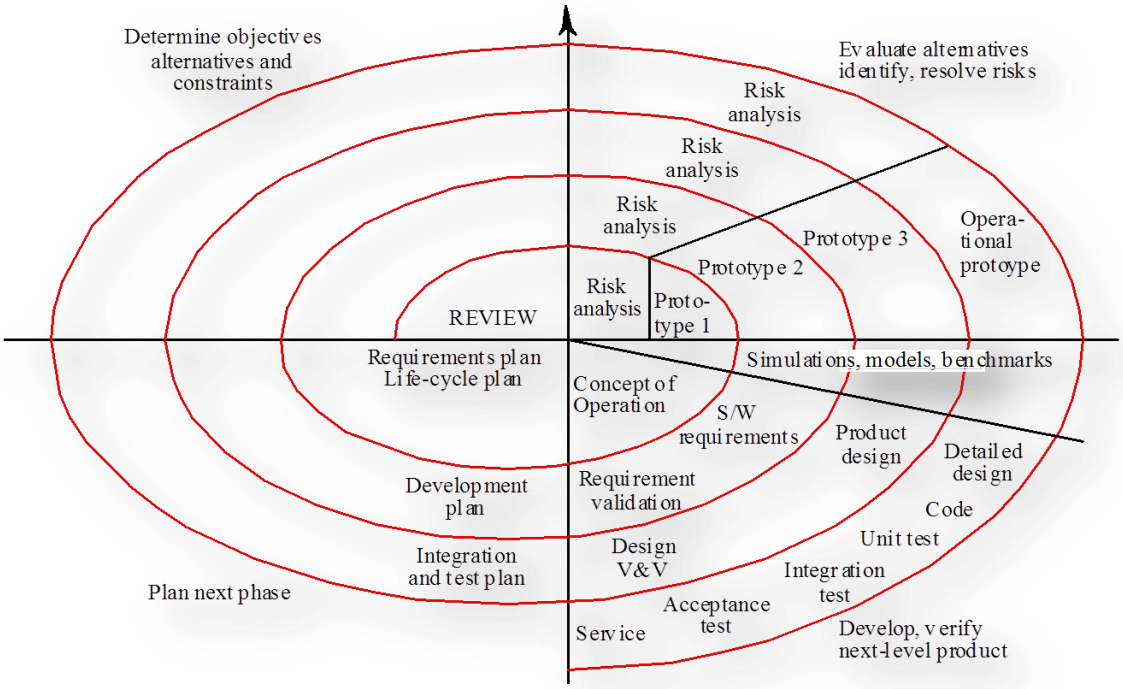
خلاصة:

النموذج التكراري يوفر إطارًا مرنًا لتطوير البرمجيات، مما يساعد الفرق على تحسين المنتجات باستمرار استنادًا إلى الملاحظات والتقييمات.

- **النموذج الحلزوني (Spiral model) :**

يمثل تطوير العمليات على هيئة لولبية بدلا من تتابع متتال للأنشطة أو التتابع المتتالي مع الرجوع عكسياً لمزيد من التحسين ، وتمثل كل حلقة من الحلزون مرحلة من مراحل العملية ، ولا توجد مراحل ثابتة مثل توصيف المتطلبات أو التصميم ، ويتم اختيار الحلقات في الحلزون بناء على ما هو مطلوب ، ويتم تقدير المخاطر وتحليلها وحل أمورها خلال العمليات .

Spiral model of the software process



- **قطاعات نموذج الحلزون :**
  - في الربع الأول من الحلزون يتم تحديد الأهداف والبدائل والقيود .
  - في الربع الثاني يتم تقييم البدائل وتعريف وتحليل المخاطر .
  - في الربع الثالث يتم التطوير والتأكد من منتج المرحلة التالية .
  - في الربع الأخير يتم تخطيط المرحلة التالية .

- **النموذج الحلزوني:(Spiral Model) :**

هي واحدة من نماذج تطوير البرمجيات التي تجمع بين نموذج الشلال ونموذج التكراري

الخصائص الرئيسية للنموذج الحلزوني:

1. **تكرار وتطوير تدريجي:**
   - يعتمد النموذج الحلزوني على تكرار المراحل، حيث يتم تطوير النظام عبر سلسلة من الدورات (spirals) التي تشمل التحليل، التصميم، التنفيذ، والتقييم.
2. **تقييم المخاطر:**
   - يتم التركيز على تقييم المخاطر في كل دورة. يتم تحديد المخاطر المحتملة وتحليلها، مما يساعد في تقليل المخاطر المرتبطة بالمشروع.
3. **تفاعل مع المستخدمين:**
   - يشمل النموذج تفاعلًا مستمرًا مع العملاء والمستخدمين للحصول على الملاحظات وتعديل المتطلبات، مما يعزز من إمكانية تلبية احتياجاتهم.
4. **مراحل النموذج:**
   - **التخطيط** :تحديد الأهداف والمتطلبات.
   - **تحليل المخاطر** :تقييم المخاطر وتحديد الحلول، وانشاء نموذج مبدئي.
   - **التطوير** :تنفيذ جزء من النظام، (تنفيذ أول المتطلبات حسب الأولوية التي حددتها مرحلة التخطيط ومرحلة تحليل المخاطر للوصول الأهداف).
   - **التقييم** :مراجعة النظام مع أصحاب المصلحة.
5. **المرونة:**
   - يسمح النموذج بتغييرات مستمرة بناءً على الملاحظات والتقييمات، مما يجعله مناسبًا للمشاريع التي تتطلب تفاعلًا مستمرًا مع المتطلبات.

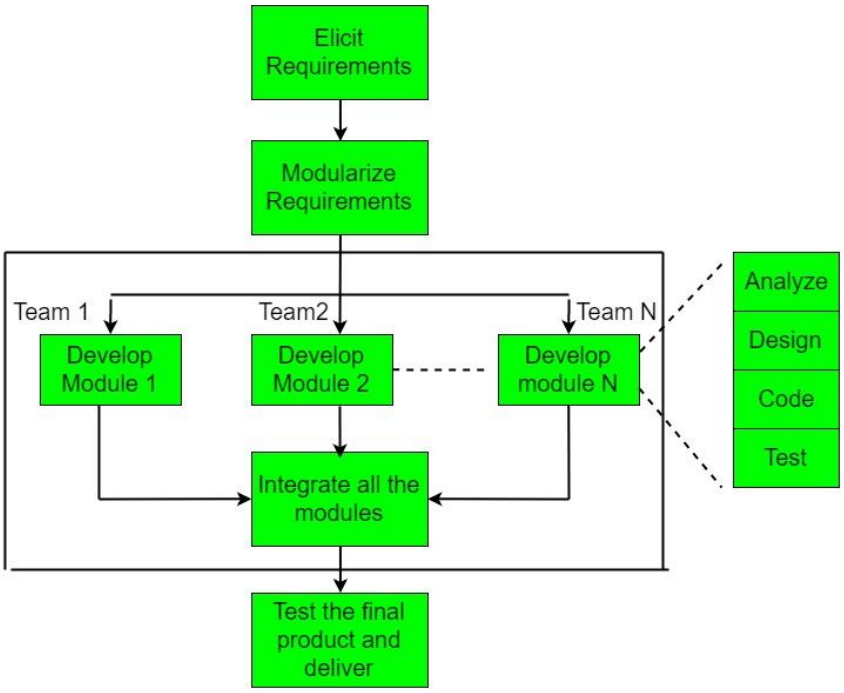
استخدامات النموذج الحلزوني:

- يُفضل استخدام النموذج في المشاريع الكبيرة والمعقدة حيث تكون المخاطر عالية، أو عندما تكون متطلبات المشروع غير واضحة في البداية.
- يُستخدم أيضًا في المشاريع التي تتطلب تغييرات مستمرة وتفاعلاً مستمرًا مع العملاء.

العيوب:

- يمكن أن يكون النموذج مكلفًا ومعقدًا، ويتطلب إدارة دقيقة لضمان نجاحه.
- قد يؤدي التركيز على تقييم المخاطر إلى تأخير في الجدول الزمني.

- **النموذج السريع (Rapid model) & (Rapid Application Development) :**



❖ **ما هي منهجية (RAD) :**

هي عملية تطوير برمجية تعتمد على النماذج الأولية دون أي تخطيط محدد.

في منهجية (RAD) يتم إعطاء اهتمام أقل للتخطيط ويتم إعطاء المزيد من الأولوية لمهام التطوير، وتهدف المنهجية إلى تطوير البرامج في فترة زمنية قصيرة.

تركز منهجية (RAD) على مصدر المدخلات والمخرجات ووجهة المعلومات ، تؤكد على تسليم المشاريع في أجزاء صغيرة، بحيث تنقسم المشاريع الكبيرة إلى سلسلة من المشاريع الأصغر. وتتمثل السمات الرئيسية لمنهجية (RAD) في أنها تركز على إعادة استخدام القوالب والأدوات والعمليات والكود.

❖ **المراحل الرئيسية لمنهجية RAD :**

نموذج العمل السريع (RAD) يتميز بتركيزه على التطوير السريع للبرمجيات من خلال دورات تطوير قصيرة ومتكررة. وتتضمن المراحل الرئيسية في نموذج العمل السريع (RAD) ما يلي:

1- تحليل المتطلبات : (Requirements Analysis) في هذه المرحلة، يتم تحديد وتحليل متطلبات النظام بالتعاون مع العملاء والمستخدمين.
2- تصميم النظام والبرمجيات :(System and Software Design) يتم في هذه المرحلة تصميم النظام والبرمجيات بناءً على المتطلبات المحددة سابقًا.
3- التنفيذ واختبار الوحدة :(Implementation and Unit Testing) يتم في هذه المرحلة تنفيذ البرمجيات واختبار كل وحدة بشكل منفصل.

4- التكامل واختبار النظام :(Integration and System Testing) يتم في هذه المرحلة دمج الوحدات المختلفة معًا واختبار النظام ككل لضمان توافقه وأدائه الجيد.

5- التشغيل والصيانة (Operation and Maintenance): تعتبر هذه المرحلة من المراحل الدائمة حيث يتم تشغيل النظام وصيانته وتحديثه بناءً على احتياجات المستخدمين والتغييرات اللاحقة.

❖ **متى تستخدم منهجية RAD :**

- عندما نحتاج إلى الانتهاء من العمل على النظام في فترة زمنية قصيرة (2-3) أشهر.
- عندما تكون المتطلبات معروفة.
- عندما يشارك المستخدم طوال دورة حياة التطوير.
- عندما تكون المخاطر الفنية أقل.
- عندما تكون هناك ضرورة لإنشاء نظام يمكن تشكيله في غضون (2-3) أشهر من الوقت.
- عندما تكون الميزانية مرتفعة بما يكفي لتحمل المصممين للنمذجة بالإضافة إلى تكلفة الأدوات الآلية لإنشاء الكود.

❖ **مزايا منهجية RAD :**

- مفيدة عندما يتعين عليك تقليل مخاطر المشروع الإجمالية.
- قابلة للتكيف ومرنة مع التغييرات.
- أسهل في نقل المخرجات حيث يتم استخدام البرامج النصية والتجريدات عالية المستوى والأكواد الوسيطة.
- بسبب مولدات الكود وإعادة استخدام الكود ، هناك انخفاض في كتابة الكود اليدوي.
- بسبب النماذج الأولية ، هناك احتمال لعيوب أقل.
- تقدم كل مرحلة في منهجية (RAD) وظائف ذات أولوية قصوى لاحتياجات لعميل.

❖ **عيوب نموذج RAD :**

- لا يمكن استخدامها للمشاريع الصغيرة.
- ليست كل التطبيقات ملائمة مع (RAD).
- عندما تكون المخاطر الفنية عالية ، فهي غير مناسبة.
- إذا لم يلتزم المطورون بتسليم البرامج في الوقت المحدد ، يمكن أن تفشل مشاريع (RAD).
- البرنامج يحتوي على ميزات قليلة بسبب ضيق الوقت ، حيث يتم ارسال الميزات إلى إصدار الأحدث لإنهاء الإصدار الحالي في فترة قصيرة.
- القابلية للتوسع منخفضة لأن منهجية (RAD) تبدأ كنموذج أولي وتتطور إلى تطبيق مكتمل.
- من الصعب تتبع التقدم والمشكلات المعتادة ، وذلك لعدم تواجد وثائق تثبت ما تم إنجازه.
- يتطلب مصممين أو مطورين ذوي مهارات عالية.

* **ما هي أفضل طريقة لاختيار نموذج تطوير مناسب !**

اختيار النموذج المناسب لتطوير البرمجيات يعتمد على عدة عوامل، بما في ذلك طبيعة المشروع، متطلبات العملاء، مدى توافق النموذج مع عملية التطوير المحددة، وقدرة الفريق على تطبيق النموذج بفعالية.

وهذه بعض الخطوات التي يمكن اتباعها لاختيار النموذج المناسب:

1- تحليل متطلبات المشروع:

- الفهم الجيد لمتطلبات المشروع وطبيعتها، وتحديد ما إذا كانت المتطلبات ثابتة أم متغيرة يمكن أن يوجهنا لاختيار النموذج المناسب.

2- تقييم الموارد والجدول الزمني:

- تقييم الموارد المتاحة، بما في ذلك الميزانية والفريق والوقت. يجب أن يكون النموذج مناسبًا لهذه الموارد.

3- فهم مستوى التغير المتوقع:

- إذا كانت هناك توقعات لتغييرات كثيرة خلال عملية التطوير، فقد تكون النماذج المرنة مثل Agile مناسبة.

4- تفضيلات العميل والفريق:

- الاستماع إلى آراء العملاء وفريق التطوير حول النماذج التي يرغبون في استخدامها والتي يشعرون بأنها ستكون مناسبة.

5- التحقق من تجارب سابقة:

- التجارب السابقة للفريق في استخدام أنماط معينة لتطوير البرمجيات وتقييم نجاحها يعطي تصور أكثر وضوحاً لاستخدام النموذج المناسب.

6- اختبار وتقييم النموذج المستهدف:

- تجربة النموذج المستهدف على نطاق صغير قبل تبنيه بشكل كامل لضمان أنه يتوافق مع احتياجات المشروع.

7- مراعاة الظروف الخاصة:

- قد تحتاج ظروف معينة إلى نموذج محدد، مثل مشروعات البحث والتطوير التي قد تستفيد من نموذج تطوير تجريبي.

وبصفة عامة فإن الاختيار الصحيح لنموذج تطوير البرمجيات يتطلب فهما جيدا لمتطلبات المشروع وظروفه وقدرات الفريق، بالإضافة إلى الاستفادة من التجارب السابقة واختبار النماذج المحتملة قبل اتخاذ القرار النهائي.

* **أهمية مرحلة اختبار النموذج قبل التطبيق الكامل:**

مرحلة اختبار النموذج (Prototype Testing) قبل تطبيقه بشكل كامل تساعد على ضمان نجاح عملية التطوير وتقديم نظام برمجيات يلبي احتياجات المستخدمين بشكل فعال وتعتبر أمرًا هاماً جداً في عملية التطوير. وهذه بعض الأسباب الرئيسية التي تجعل من هذه المرحلة أساسية:

1. **تحديد الاحتياجات بشكل أفضل:**

- يمكن لاختبار النموذج مساعدة الفريق على فهم الاحتياجات بشكل أفضل من خلال تفاعل المستخدمين مع نموذج النظام المبدئي.

2. **توضيح التصميم والوظائف:**

- يساعد اختبار النموذج في توضيح التصميم والوظائف المقترحة للنظام، مما يسمح بتحديد أي تعديلات أو تحسينات قبل البدء في التطبيق الكامل.

3. **تقليل المخاطر وتوفير التكاليف:**

- بفضل اختبار النموذج، يمكن اكتشاف المشاكل والعيوب في وقت مبكر من دورة التطوير، مما يقلل من المخاطر ويوفر التكاليف لإجراء التغييرات.

4. **زيادة تفاعل العملاء:**

- يمكن لاختبار النموذج أن يزيد من تفاعل العملاء ويساعدهم في فهم كيف سيعمل النظام النهائي، مما يقلل من احتمالية حدوث سوء التفاهم.

5. **تسهيل عملية التواصل:**

- يعمل النموذج كأداة تواصل فعالة بين فريق التطوير والعملاء أو المستخدمين، مما يساعد على تحقيق توافق أفضل بشأن متطلبات النظام.

6. **تحسين تجربة المستخدم:**

- من خلال اختبار النموذج، يمكن تحسين تجربة المستخدم وضمان أن النظام النهائي يلبي احتياجاتهم وتوقعاتهم.